

CS50P - Lecture 6: File I/O (Master Notes)

1. The Core Problem: Data Volatility (RAM vs. Disk)

Up until this point in the course, any data stored in our programs has lived exclusively in **Variables** (such as integers, strings) or **Data Structures** (like `list` or `dict`).

The Physical Limitation of In-Memory Storage

To understand why we need File I/O, we must look at how a computer physically manages data across two distinct components:

- **RAM (Random Access Memory):** This is where Python stores variables and lists while a program is actively running. RAM is incredibly fast, but it is **volatile**. It requires constant electrical power to maintain data. The moment a Python program finishes running or the computer resets, the operating system reclaims that memory, and your data **completely disappears**.
- **Disk Storage (HDD / SSD):** This is your computer's permanent file system. It is **non-volatile**, meaning data stays intact even when the power is completely shut off.

Key Takeaway: If you want your program to remember information across different sessions (e.g., saving user profiles, high scores, or logs), storing data in standard RAM variables is not enough. We need File I/O to move data out of temporary RAM and safely onto the permanent Disk.

2. Flat-File Storage: The Simplest Approach

To make data **persistent**, we interact with plain text files on the disk.

Demonstration: The Memory-Only Version

In an initial script (`names.py`), we collect names and store them in a list:

Code (`names.py`):

```
Python
names = []

for _ in range(3):
    names.append(input("What's your name? "))

for name in sorted(names):
    print(f"hello, {name}")
```

Terminal Window:

```
Plaintext
$ python names.py
What's your name? Hermione
What's your name? Harry
What's your name? Ron
hello, Harry
hello, Hermione
hello, Ron
```

Concept Spotlight: Deep-Dive into `.append()`

The `.append()` function is a built-in method specifically used with Python lists to add an item to the very end of an existing list.

- **In-Place Modification:** It alters the original list directly in RAM. It does *not* create a copy or return a new list.
- **Under the Hood:** Python locates the next empty memory slot at the end of the array (Index: n), inserts the single data object, and increments the list's total length counter by 1.

Deep Dive: Printing the Iterator vs. Printing the List Container

When experimenting with inline printing inside the data collection loop, a critical distinction arises based on whether you call the collection container or an unassigned singular variable.

Case A: The Unassigned Singular Variable (Crash)

Code:

```
Python
names = []
for _ in range(3):
    names.append(input("What's your name? "))
    print(f"hello, {name}")
```

Terminal Window:

```
Plaintext
$ python names.py
What's your name? amit
Traceback (most recent call last):
  File "names.py", line 4, in <module>
    print(f"hello, {name}")
NameError: name 'name' is not defined
```

- **Why it fails:** `input()` passes its value directly into `.append()`. The singular variable `name` is never declared or bound to a value in memory.

Case B: Printing the Entire List Container (Successful Run)

Code:

```
Python
names = []
for _ in range(3):
    names.append(input("whats your name "))
    print(f"hello,{names}")
```

Terminal Window:

```
Plaintext
whats your name amit
hello,['amit']
whats your name
hello,['amit', '']
whats your name jack
hello,['amit', '', 'jack']
```

- **Why it works:** `names` is a valid, declared list object.
- **Behavior:** Instead of extracting individual strings, Python prints the literal string representation of the array data structure at that exact moment in execution. If the user presses return without entering text, an empty string (") is appended, showing the literal state of RAM.

3. Writing to Files using `open()`

To transition from RAM to permanent storage, Python provides a built-in function called `open()`. This function establishes a direct line of communication between the program and the computer's file system.

Code (`names.py`):

```
Python
name = input("What's your name? ")

file = open("names.txt", "w")
file.write(name)
file.close()
```

Breakdown of the Syntax

- `open("filename", "mode")`: This function takes two main arguments:
 1. **Filename:** The name of the file you want to create or alter (e.g., `"names.txt"`).
 2. **Mode:** A string that defines what you intend to do with the file.

- **file.write(data)**: A method used to place text inside the open file.
- **file.close()**: This explicitly tells the operating system that your program is finished with the file, forcing all cached data to save ("flush") to the disk and releasing the file lock.

4. The Overwrite Problem & Audience Interaction

After demonstrating the code above, the program is run twice in succession via the terminal:

Terminal Window:

Plaintext

```
$ python names.py
```

```
What's your name? Harry
```

```
$ python names.py
```

```
What's your name? Ron
```

File Content Window (**names.txt**):

Plaintext

Ron



Audience Interaction: The Overwrite Mystery

Audience Member: Why did Harry disappear from our file when we ran the program a second time to add Ron?

David Malan's Answer: The culprit is the second argument passed to the open function: **"w"** (Write Mode). Every single time Python executes **open("names.txt", "w")**, it systematically deletes the pre-existing file of that name if it exists, creates a brand-new empty file, and starts writing from scratch. It does *not* mix or add to past text data.

5. Shifting to Append Mode ("**a**") and the Newline Fix

To allow data to accumulate over time instead of deleting past records, we switch from write mode to **append mode ("a")**.

The Formatting Catch: **file.write()** vs. **print()**

Unlike Python's standard **print()** function—which automatically jumps to a new line after printing—the **file.write()** method does **not** add any formatting automatically.

If you run the script three times back-to-back using just **file.write(name)** without a newline character:

Terminal Window:

Plaintext

```
$ python names.py
```

```
What's your name? Hermione
```

```
$ python names.py
```

```
What's your name? Harry
```

```
$ python names.py
```

```
What's your name? Ron
```

File Content Window (**names.txt**):

Plaintext

```
HermioneHarryRon
```

The text file wraps them all horizontally into a single, unreadable word string because no line breaks tell the computer to step down.

The Fix: Integrating **\n**

To keep data neatly organized, we pass a formatted string literal containing a newline escape character (**\n**). This instructs the cursor to drop down to a brand-new line immediately after saving the current name.

Code (**names.py**):

Python

```
name = input("What's your name? ")
```

```
file = open("names.txt", "a")
```

```
file.write(f"{name}\n")
```

```
file.close()
```

The Corrected Result

By running the updated script with **\n** across multiple sessions:

Terminal Window:

Plaintext

```
$ python names.py
```

```
What's your name? Hermione
```

```
$ python names.py
```

```
What's your name? Harry
```

```
$ python names.py
```

```
What's your name? Ron
```

File Content Window (**names.txt**):

Plaintext
Hermione
Harry
Ron

6. The Modern/Pythonic Way: The **with** Block

While explicitly closing files using `file.close()` works, it introduces a major vulnerability. If your code encounters an error or crashes *before* reaching the `.close()` statement, the file remains open and locked in system memory. This can lead to corrupted data or file conflicts.

Python provides a more elegant, safer syntax to automate this cleanup using the `with` keyword, which leverages a concept known as a **Context Manager**.

Code (**names.py**):

```
Python
name = input("What's your name? ")

with open("names.txt", "a") as file:
    file.write(f"{name}\n")
```

Why Use **with**?

- **Defines a Block Structure:** The file is opened and assigned to a variable moniker (as `file`) exclusively for use within the indented block.
- **Automatic Resource Cleanup:** The moment the program exits the indented block (either because it finished running the lines or because a crash occurred), Python implicitly shuts down and flushes the file behind the scenes. You no longer write `file.close()`.

7. Reading Files: **"r"** Mode & **.readlines()**

Once text records are permanently stored inside `names.txt`, the focus shifts to bringing that data back into your application. To do this, we use the file function in **Read Mode ("r")**. Note that if you leave out the mode argument entirely, Python defaults to `"r"`.

Step 1: Loading lines via **.readlines()**

The `.readlines()` method targets an open file, extracts every text row, and bundles those rows together as individual string items inside a list.

Code (**names.py**):

```
Python
with open("names.txt", "r") as file:
    lines = file.readlines()

for line in lines:
    print("hello,", line)
```

The Newline Spacing Glitch

When executing this read routine on our file, an unexpected layout visual appears in the shell:

Terminal Window:

```
Plaintext
$ python names.py
hello, Hermione

hello, Harry

hello, Ron
```



Audience Interaction: The Double Newline Mystery

Audience Member: Why are there empty blank lines separating each greeting in our output loop?

David Malan's Answer: Each line inside **names.txt** explicitly holds an invisible newline character (`\n`) that we intentionally appended earlier. When `.readlines()` reads the file, it captures those hidden flags, making the loop strings look like `"Hermione\n"`. When you pass that string into `print()`, two consecutive line drops happen: one triggered by the `\n` inside the file data, and a second triggered automatically by `print()`'s default behavior.

The Fix: Applying **.rstrip()**

To resolve this formatting glass ceiling, we utilize a string method called `.rstrip()` (Right Strip). This removes any trailing whitespace or newline characters from the right side of a string data value.

Code (**names.py**):

```
Python
with open("names.txt", "r") as file:
    lines = file.readlines()
```

```
for line in lines:
    print(f"hello, {line.rstrip()}")
```

Terminal Window:

```
Plaintext
$ python names.py
hello, Hermione
hello, Harry
hello, Ron
```

8. Optimizing Code: Iterating the File Object Directly

While using `.readlines()` works perfectly for short files, it forces Python to copy every single row of your document into a list in RAM at once. If your file is gigabytes in size, this can completely freeze your computer's memory.

Instead of creating an intermediate list container, Python allows you to iterate directly over the file pointer object inside a loop structure.

Code (`names.py`):

```
Python
with open("names.txt", "r") as file:
    for line in file:
        print(f"hello, {line.rstrip()}")
```

Terminal Window:

```
Plaintext
$ python names.py
hello, Hermione
hello, Harry
hello, Ron
```

Under the Hood

- This approach keeps your code incredibly concise.
 - By treating the open file as an iterable sequence, Python efficiently streams one row into memory at a time, processes the `print()` statement, discards that line from RAM, and moves on to the next.
-

9. Sorting Data Read from a File

In the baseline example of reading from a file, the names are printed exactly in the order they were written on the disk. To sort them alphabetically, we stream them back into an internal list container in RAM first.

Code (`names.py`):

```
Python
names = []

with open("names.txt") as file:
    for line in file:
        names.append(line.rstrip())

for name in sorted(names):
    print(f"hello, {name}")
```

Terminal Window:

```
Plaintext
$ python names.py
hello, Draco
hello, Harry
hello, Hermione
hello, Ron
```

10. Student Discovery: Read-Loop Mode Mismatch Execution

An instructive logical bug occurs when a developer attempts to read from a file descriptor that has been structurally configured strictly for data writing output formats.

The Bug Code:

```
Python
names = []
with open("names.txt", "w") as file:
    for line in file:
        names.append(line.rstrip())
for name in sorted(names):
    print(f"hello,{names}")
```

Terminal Window:

Plaintext

```
amitkumar@amits-MacBook-Air-2 python lecture 6 % /usr/local/bin/python3 "names.py"
```

```
amitkumar@amits-MacBook-Air-2 python lecture 6 %
```

Error Isolation Analysis

1. **The Silent File Erasure:** Opening `names.txt` with `"w"` (Write Mode) automatically clears any pre-existing text data on disk. The file is empty at the absolute start of the `with` block scope.
2. **Empty Loop Skipping:** Because the text target contains `$0$` bytes, the internal stream statement `for line in file:` determines there are zero sequences to process. It completely skips the indented block lines underneath it, leaving the `names` array list object entirely empty (`[]`).
3. **Empty Loop Output Execution:** The closing block `for name in sorted(names):` attempts to run against an empty list sequence. Since there are no elements to unpack, the printing routine never engages, completing execution with no output characters or crash tracking logs.

11. Storing Multi-Column Data: Introduction to CSVs

As scripts scale, tracking a single unstructured variable (like just a string name) becomes insufficient. Programs often require managing associated metadata blocks (e.g., tracking both a student's name *and* their matching school house).

To transition away from a single plain row, David Malan introduces **CSV (Comma-Separated Values)** flat-file storage layouts.

Structuring a Multi-Column Flat File

Instead of storing values isolated line-by-line, each row represents an entire entity record, with individual fields separated by a literal comma marker delimiter.

File Content Window (`students.csv`):

Plaintext

Hermione,Gryffindor

Harry,Gryffindor

Ron,Gryffindor

Draco,Slytherin

Basic Parsing Using `.split()`

To read and break down this structurally split data file format, Python strings come equipped with a built-in method called `.split()`.

When you pass a specific boundary character into `.split("boundary")`, it dissects the single string object at every instance of that boundary and divides it into a sequence of separate items stored in a temporary list.

Code (students.py):

```
Python
with open("students.csv") as file:
    for line in file:
        row = line.rstrip().split(",")
        print(f"{row[0]} is in {row[1]}")
```

Terminal Window:

```
Plaintext
$ python students.py
Hermione is in Gryffindor
Harry is in Gryffindor
Ron is in Gryffindor
Draco is in Slytherin
```

Code Architecture Under the Hood

1. `line.rstrip()`: Strips away the invisible hidden line jump flags (`\n`) flanking the right edge of each file entry row.
2. `.split(",")`: Searches the active line sequence string for a comma character, splits it, and turns the single text record row into a neat, indexed two-item list array: `["Hermione", "Gryffindor"]`.
3. **Unpacking Index Pointers**: The print command maps out positional index items, pulling `row[0]` to populate the name segment and `row[1]` to populate the house component.

12. Pythonic Optimization: Unpacking Variables Inline

Relying on direct variable numerical list indexes like `row[0]` or `row[1]` can easily lead to code that is confusing and difficult to read. Python enables developers to unpack the list directly into descriptive, custom variable references inline within a single code line assignment block.

Code (students.py):

```
Python
with open("students.csv") as file:
    for line in file:
        name, house = line.rstrip().split(",")
        print(f"{name} is in {house}")
```

Terminal Window:

Plaintext

```
$ python students.py
```

```
Hermione is in Gryffindor
```

```
Harry is in Gryffindor
```

```
Ron is in Gryffindor
```

```
Draco is in Slytherin
```

Strict Syntax Integrity Warning

When utilizing Python's variable unpacking feature, the total number of variable structures positioned on the left side of your operational assignment symbol must **exactly match** the exact count of sliced item blocks generated by the string split operation on the right side.

- If `line.split(",")` generates exactly two data parts, but you provide only a single variable identifier loop layout (e.g., `name = line.rstrip().split(",")`), the variable will capture the entire two-item list slice array (`['Hermione', 'Gryffindor']`).
- If `line.split(",")` yields two split segments, but your code provides three separate targets (e.g., `name, house, age = line.rstrip().split(",")`), your program execution will immediately halt and raise a **ValueError: too many values to unpack**.

13. Sorting Multi-Column Strings inside a List

Once we introduce `students.csv`, David Malan notes that if we want to sort the rows alphabetically, we can run into a layout challenge if we just blindly pull strings.

As an initial optimization step before using advanced structures, he shows how to construct complete string sentences and append *those* right into our sorting list container.

Code (`students.py`):

Python

```
students = []
```

```
with open("students.csv") as file:
```

```
    for line in file:
```

```
        name, house = line.rstrip().split(",")
```

```
        students.append(f"{name} is in {house}")
```

```
for student in sorted(students):
```

```
    print(student)
```

Terminal Window:

Plaintext

```
$ python students.py
```

Draco is in Slytherin

Harry is in Gryffindor

Hermione is in Gryffindor

Ron is in Gryffindor

The Technical Need for `students.append()`

- **Overcoming File Stream Limitations:** Python reads the file line-by-line. Once the loop steps to a new line, it completely forgets the previous one.
- **The List as a "Waiting Room":** Because `sorted()` requires seeing all elements simultaneously to organize them, `students.append()` stores each row inside our empty `students = []` list container in RAM. This builds the complete collection row-by-row before sorting can begin.

The Downside to This Approach

While this technique prints a beautifully sorted list, it locks our data into a single formatted sentence string inside RAM. If we later wanted to change the phrase to *"Gryffindor welcomes Harry"* or sort specifically *only* by the House names instead of student names, this raw string list method fails. We have completely lost the fine-grained control over our individual columns.

14. Sorting Multi-Column Data Using Dictionaries

To retain individual control over each column while keeping them grouped as a single object, we transition from plain strings to individual dictionary (`dict`) objects before appending them to our master tracking list.

Step 1: Accumulating Composite Rows as Dictionary Lists

Code (`students.py`):

Python

```
students = []
```

```
with open("students.csv") as file:
```

```
    for line in file:
```

```
        name, house = line.rstrip().split(",")
```

```
        student = {"name": name, "house": house}
```

```
        students.append(student)
```

```
for student in students:
```

```
    print(f'{student["name"]} is in {student["house"]}')"
```

Terminal Window:

Plaintext

```
$ python students.py
```

Hermione is in Gryffindor

Harry is in Gryffindor

Ron is in Gryffindor

Draco is in Slytherin

Step 2: The Sorting Challenge (Passing Custom Functions)

If you try to simply execute `sorted(students)` on this new tracking object list array, Python will crash instantly with a `TypeError`. This happens because Python does not inherently know *how* to compare two dictionaries (should it sort them by *name*? By *house*?).

To resolve this, the `sorted()` function accepts an optional keyword parameter argument named `key`. We can pass a custom function into `key` that specifically instructs Python exactly which dictionary entry field key structure it should use for ordering comparisons.

Code (`students.py`):

Python

```
students = []
```

```
with open("students.csv") as file:
```

```
    for line in file:
```

```
        name, house = line.rstrip().split(",")
```

```
        student = {"name": name, "house": house}
```

```
        students.append(student)
```

```
def get_name(student):
```

```
    return student["name"]
```

```
for student in sorted(students, key=get_name):
```

```
    print(f'{student["name"]} is in {student["house"]}')"
```



Audience Interaction: Sorting by Key Variations

Audience Member: If we wanted to sort our list of dictionaries by their House names instead of the student names, how would we change this?

David Malan's Answer: All you would need to do is modify the targeted dictionary key inside your custom sorting function. Instead of returning `student["name"]`, you change the function to return `student["house"]`. Python's `sorted()` function will immediately switch from tracking student alphabetical order to grouping and sorting the rows by their respective Hogwarts houses.

15. Streamlining Dictionaries with Anonymous Lambda Functions

While passing a custom function like `key=get_name` works perfectly, writing an entire `def` code block just to return a single key inside a dictionary can make scripts verbose. Python provides a way to write quick, inline functions without a name, known as **Lambda Functions**.

What is a Lambda Function?

A lambda function is a small, anonymous function that can be written on a single line. It is used when you need a throwaway function for a very short period—most commonly as a sorting key.

Refactoring to use Lambdas

Code (`students.py`):

Python

```
students = []
```

```
with open("students.csv") as file:
```

```
    for line in file:
```

```
        name, house = line.rstrip().split(",")
```

```
        student = {"name": name, "house": house}
```

```
        students.append(student)
```

```
for student in sorted(students, key=lambda student: student["name"]):
```

```
    print(f'{student["name"]} is in {student["house"]}')"
```

Breakdown of the Lambda Syntax

Let's dissect the expression passed into the key parameter: `key=lambda student: student["name"]`

- **lambda**: This keyword tells Python, *"I am defining a quick, anonymous function right here on this line."*
- **student** (before the colon): This is the input parameter. It acts exactly like the parameter inside `def get_name(student):`. As `sorted()` steps through the list, it passes each dictionary into this variable.
- **student["name"]** (after the colon): This is the expression that gets evaluated and **automatically returned**. You do *not* write the word `return`; lambdas implicitly return whatever value this expression produces.

Why do we use Lambda? (Deep Dive Explanation)

Lambda functions solve a clutter problem. Without a lambda, if you want to sort a list of dictionaries by a specific value, you are forced to write a separate standalone function using `def`. This approach has two major downsides:

1. **Name Pollution:** You create a function name (`get_name`) that lives in your global program scope, even though you only intend to use it once on a single line of code.
2. **Code Disconnection:** A reader has to look away from your `sorted()` routine, find where `get_name` was defined elsewhere in the file, and match them up visually.

By using `lambda`, you write the logic exactly where you need it, avoiding the need to name a function you'll never use again.

Sorting in Reverse Order

The `sorted()` function also accepts another useful named argument: `reverse`. By setting `reverse=True`, we can instantly invert the final sequence layout from Z to A.

Python

```
# Sorting alphabetically in descending order
```

```
for student in sorted(students, key=lambda student: student["name"], reverse=True):
```

16. Reading CSV Files via Python's Built-In `csv` Module

Up to this point, our custom manual tool for tracking files has been `.split(",")`. This works perfectly as long as our data sheets are simple and pristine. However, standard comma splitting faces a fatal flaw when handling real-world data layouts.

The Breakdown Scenario: The Boundary Problem

Imagine one of our students has an address or a formal name format that includes a literal comma inside it (e.g., Harry's home address is written as `"Gryffindor, Tower"`).

File Content Window (`students.csv`):

Plaintext

```
Harry,"Gryffindor, Tower"
```

If we execute our manual line parser: `name, house = line.rstrip().split(",")`

- Python encounters **two** commas instead of one.
- It slices the row into three distinct pieces (`"Harry"`, `"Gryffindor"`, `' Tower'`).
- Because our code only assigned two variables (`name`, `house`), the program immediately crashes with a `ValueError: too many values to unpack`.

The Solution: `csv.reader`

To handle these hidden anomalies gracefully, Python provides an official standard library module named `csv`. Rather than manually splitting lines as generic strings, we pass our file object into `csv.reader()`.

Code (`students.py`):


```
Python
import csv

students = [ ]

with open("students.csv") as file:
    reader = csv.reader(file)
    for row in reader:
        students.append({"name": row[0], "house": row[1]})

for student in sorted(students, key=lambda student: student["name"]):
    print(f'{student["name"]} is from {student["house"]})'
```

Under the Hood Rules

- The `csv.reader` engine automatically recognizes standard CSV design constraints, such as surrounding items with quotation marks ("...") when they contain active commas.
- It intelligently groups "Gryffindor, Tower" into a single comprehensive field index block rather than dividing it at the inner boundary, preventing application crashes.

17. Improving Code Clarity: `csv.DictReader`

While `csv.reader` protects our code from crashing, indexing data via `row[0]` and `row[1]` requires us to blindly guess which index column correlates to which data attribute. If someone changes the structure of our CSV sheet by shifting the house column to index position zero, our code breaks down internally.

To write completely robust data scripts, we introduce a secondary optimization engine from the module: `csv.DictReader`.

The Prerequisite: Header Rows

For a `DictReader` structure to execute flawlessly, our CSV flat-file must lead with an explicit **header block row** that assigns structural field label definitions to each vertical column segment.

File Content Window (`students.csv`):

```
Plaintext
name,house
Hermione,Gryffindor
Harry,Gryffindor
Ron,Gryffindor
```

Implementing `DictReader`

Code (`students.py`):

```

Python
import csv

students = []

with open("students.csv") as file:
    reader = csv.DictReader(file)
    for row in reader:
        students.append({"name": row["name"], "house": row["house"]})

for student in sorted(students, key=lambda student: student["name"]):
    print(f'{student["name"]} is in {student["house"]}')

```

Architectural Key Concepts

- **Dynamic Conversions:** When `csv.DictReader` loads the document, it dynamically intercepts the very first row sequence (`name,house`) and establishes those values as explicit structural key names.
- **Dictionary Mapping:** During each loop pass, the object variable `row` does *not* arrive as a sequence list array. Instead, it arrives pre-packaged as an active dictionary object: `{"name": "Hermione", "house": "Gryffindor"}`.
- **Positional Independence:** We can now call information intuitively via explicit column labels like `row["name"]`. Even if someone introduces five new middle data columns to our spreadsheet later, our specific assignments remain completely intact.

18. Writing to CSV Files via `csv.writer`

Just as we can read complex structural rows with the help of the standard library, Python provides tools to write clean, formatted multi-column data directly to a `.csv` file. This lets us capture multiple distinct inputs from a user and securely store them as single comma-separated rows.

Implementing `csv.writer`

When saving multi-column data, we initialize a writer object by passing our file stream pointer directly into `csv.writer()`. We then use the `.writerow()` method to pass an explicit sequence list representing our columns.

Code (`students.py`):

```

Python
import csv

name = input("What's your name? ")
home = input("Where's your home? ")

```

with open("students.csv", "a") as file:

```
writer = csv.writer(file)
writer.writerow([name, home])
```

Under the Hood Rules

- **Automatic Formatting:** Much like `csv.reader`, `csv.writer` handles real-world spacing constraints for you. If a user inputs a name or a home that features literal comma characters (e.g., `Number Four, Privet Drive`), the writer object intelligently encapsulates the entire string block in quotation marks ("...") when saving it to the disk.
- **Array-to-Row Mapping:** `.writerow()` takes a flat list/array and flattens it horizontally, adding standard commas in between the items automatically.

19. The Ultimate Cleanliness: `csv.DictWriter`

To achieve the highest degree of structural stability when generating entries, we switch from hardcoded lists to dictionary rows using `csv.DictWriter`. This ensures your code remains clean by forcing you to state column positions explicitly using descriptive structural dictionary keys.

Implementing `csv.DictWriter`

When creating a `DictWriter`, we must supply a secondary required keyword argument called `fieldnames`. This parameter accepts a precise list/array stating the exact order of keys we expect our incoming data dictionaries to possess.

Code (`students.py`):

Python

```
import csv
```

```
name = input("What's your name? ")
home = input("Where's your home? ")
```

with open("students.csv", "a") as file:

```
writer = csv.DictWriter(file, fieldnames=["name", "home"])
writer.writeheader() # Writes the top row column headers if creating a brand new file
writer.writerow({"name": name, "home": home})
```



Audience Interaction: The Missing Header File Bug

Audience Member: What happens if we run this script repeatedly using `writer.writeheader()` while our file is opened in Append ("a") mode?

David Malan's Answer: If you place `writer.writeheader()` inside an append context manager without any condition checks, your program will write a brand-new row of

string headers (`name,home`) every single time a user logs a unique record. To avoid creating a messy spreadsheet scattered with duplicate title blocks, you should write the header row exclusively when initializing a completely blank or fresh file.

20. Introduction to Binary Files and Images (The Pillow Library)

Our entire focus up to this stage has been strictly on text files (structured as rows of strings where human-readable characters populate the files). However, computers also heavily rely on **Binary Files**—files comprised of raw 0s and 1s that represent more complex data architectures such as executable apps, audio clips, and graphical images.

To work with graphic images, Python developers rely on third-party libraries. One of the most popular packages for image processing is **Pillow** (forked from an older library known as PIL).

Installing the Pillow Library

Because Pillow is an external third-party dependency, it does not come pre-installed in Python's standard distribution. To download it to your local environment, you use Python's package manager, `pip`, via your terminal:

```
Plaintext
$ pip install Pillow
```

Reading and Writing Graphics Data

David Malan demonstrates how we can write a Python program that reads a sequence of images passed as command-line arguments and compiles them together into an animated `.gif` file.

Code (`costume.py`):

```
Python
import sys
from PIL import Image

images = []

# Loop through image files passed via command line args (skipping script name at index 0)
for arg in sys.argv[1:]:
    image = Image.open(arg)
    images.append(image)

# Save the primary image and append the remaining frames to create an animated GIF
if images:
```

```

images[0].save(
    "costumes.gif",
    save_all=True,
    append_images=images[1:],
    duration=200,
    loop=0
)

```

Breakdown of the Image Syntax

- **Image.open(arg)**: This is Pillow's explicit version of Python's built-in `open()`. Instead of returning text lines, it reads the complex binary headers of graphic file types (like `.jpeg` or `.png`) and processes them as a structured image object in memory.
- **images[0].save(...)**: We call the `.save()` method on our first frame object, providing a series of specific keyword parameters to write the remaining images out to disk:
 - **save_all=True**: Instructs Pillow to write all the image frames in our list into a single file container instead of saving just the first image.
 - **append_images=images[1:]**: Takes a sliced copy of our list array containing every image starting from index 1 onward and stacks them sequentially behind the first frame.
 - **duration=200**: Sets the display runtime speed of each individual frame sequence to exactly 200 milliseconds before transitioning to the next frame.
 - **loop=0**: Instructs the GIF encoder to loop the animation continuously infinitely.

21. Summary of File I/O Operations & Best Practices

To close out Lecture 6, David Malan reviews the core mental models required to select the correct approach for interacting with different kinds of data. Choosing the right tool depends entirely on your target data layout and file scale.

Comprehensive File Mode Comparison Matrix

Mode Keyword	Mode Full Name	Read / Write Capability	File Cursor Behavior	Side Effects / Risks
<code>"r"</code> (Default)	Read	Read Only	Placed at the absolute beginning of the file.	Will throw a <code>FileNotFoundError</code> if the file doesn't exist on disk.

"w"	Write	Write Only	Placed at the absolute beginning of the file.	Destructive. Overwrites and erases all contents of an existing file immediately.
"a"	Append	Write Only	Placed at the absolute end of the file.	Safe for adding data records over time. Automatically creates the file if it doesn't exist.
"rb" / "wb"	Binary Read / Write	Byte Stream Only	Matches positions of "r" and "w".	Used for processing non-text structures (images, video, compiled apps).

Architectural Design: The Evolution of File Extraction

The progression of File I/O techniques in Python scales systematically from primitive raw string methods to highly abstracted module components:

Plaintext

[Plain Text File] —> `line.split(",")` —> `csv.reader()` —> `csv.DictReader()`
(Simplest Format) (Fragile Parser) (Handles Commas) (Header Key Mapping)

1. **Manual String Splitting (`.split(",")`):** Highly vulnerable to structural data breaks if user input strings naturally contain punctuation boundaries like literal commas.
2. **The `csv.reader` Parser:** Protects program execution paths by wrapping internal text fields in safe escaping quotation marks.
3. **The `csv.DictReader` Component:** Represents the gold standard for text-based flat file management. It converts column blocks directly into addressable dictionary key references, meaning your code logic won't break even if columns are reordered or moved.

Final Summary Checklist

- Always prioritize the `with open(...)` context manager structure over manual `.close()` sequences to guarantee system memory resources flush correctly during crashes.
- Avoid calling heavy extraction operations like `.readlines()` into explicit list allocations if you are tracking large metrics sheets; stream over the file structure directly utilizing the `for line in file:` structural block instead.

- Utilize **Lambda expressions** (`lambda element: element["key"]`) inline within standard operational loops to keep your data sorting implementations clean and readable without cluttering your global scope with tiny, single-use functions.